



Machine Learning II

Data Science M.Sc.

Prof. Dr. Steffen Wagner
Berliner Hochschule für Technik

Winter term 2025/26



Non-linear regression methods I



Introduction

Many regression models in ML I including ridge regression and the lasso are linear supervised learning models. Often the assumptions used in a linear model are inappropriate and we should fit a more flexible function using non-linear modelling methods.

We start off with the example of polynomial regression, and use this as a starting point to develop spline regression, smoothing splines and local regression.

James et. al covers polynomial regression, step functions and basis function in detail. We will look only briefly at these as a means to developing the methods used in practice.



Keep it simple (in the beginning)

- ▶ All the methods we discuss in the beginning just use one predictor variable x .
- ▶ Generalised additive models (GAMs) adapt these ideas to multiple predictor variables (covered next week).
- ▶ The aim is to find a function $f(x)$ which fits the data

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

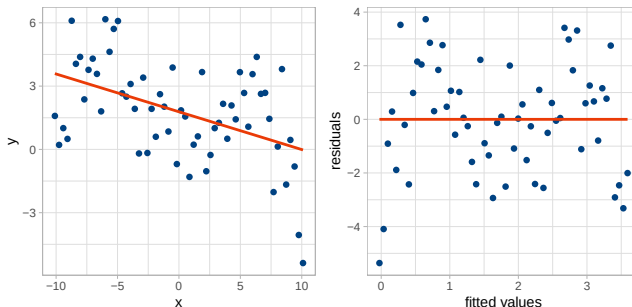
well, although there might be a non-linear relationship between y and x .

- ▶ In linear regression the predictor function is $f(x) = \beta_0 + \beta_1 x$ and we choose the coefficients β_0 and β_1 to fit the data the best that a straight line can fit the data.



Example data

Let's consider the following example data:

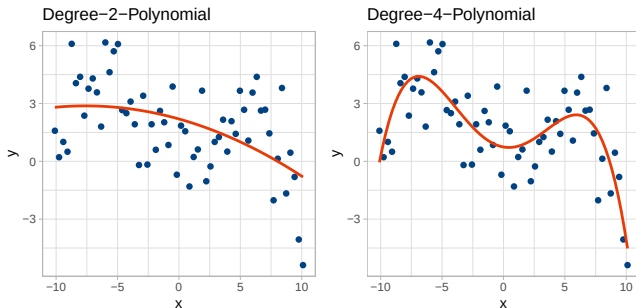


A linear regression clearly doesn't work for these data, and there is noticeable structure in the residual plot (right).



Polynomial fit

An *obvious* idea is to increase the degree of polynomial until you get a decent fit.



A quadratic regression (left) is almost exactly the same as the linear regression. Quartic regression (right) fits these data fairly well.



Polynomial regression

General formulation:

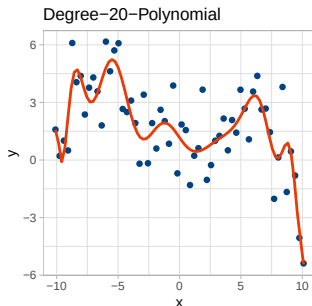
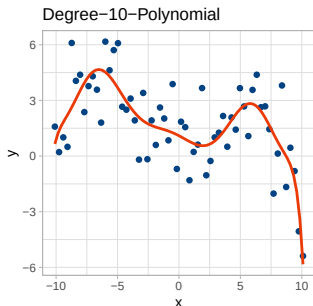
$$y_i = f(x_i) + \epsilon_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \dots + \beta_d x_i^d + \epsilon_i = \sum_{j=0}^d \beta_j x_i^j + \epsilon_i.$$

- ▶ Polynomial regression is a linear model and fitted to non-linear relationships.
- ▶ Linearity refers to a linear relationship between y and the coefficients β_j .
- ▶ Therefore polynomial models can be fitted in R using `lm()`



Polynomial regression: risk of overfitting

In practice fitting a high order polynomial $d > 3$ is rarely a good idea. In general a polynomial of order d has $d - 1$ turning points so the the polynomial curve can become overly flexible and can take on some very strange shapes. This is especially true near the boundary of the X variable.





Global vs. local relationships

- ▶ Using a polynomial function to fit the features in the data imposes a global structure on the non-linear function of X .
- ▶ We will now consider fitting several *local* functions.
- ▶ Local functions are zero outside a certain window.



Step functions

We will start by using **step functions**, which are constant within the window.

- ▶ We break the range of x into *bins*, and fit a different constant in each bin.
- ▶ This amounts to converting a continuous variable into an ordered categorical variable.
- ▶ We choose $K-1$ cut points $c_1, c_2, \dots, c_{K-1}$ within in the range of x , and fit K step functions.



Window function

The window functions is defined as

$$C_1(x) = I(x < c_1),$$

$$C_2(x) = I(c_1 \leq x < c_2), \text{ etc.}$$

$I(\cdot)$ is an *indicator function*, that returns 1 if the condition is true and returns 0 otherwise. The formal definition of e.g. $I(c_1 \leq x < c_2)$ is

$$I(c_1 \leq x < c_2) = \begin{cases} 1 & c_1 \leq x < c_2 \\ 0 & \text{otherwise.} \end{cases}$$

A piecewise constant predictor function has the form

$$f(x) = \beta_1 C_1(x) + \beta_2 C_2(x) + \cdots + \beta_K C_K(x).$$



OLS with step functions

We can now find the least squares estimates of the model

$$y_i = f(x_i) + \epsilon_i = \beta_1 C_1(x_i) + \beta_2 C_2(x_i) + \cdots + \beta_K C_K(x_i) + \epsilon_i,$$

where ϵ_i are the residuals and $\beta_1, \dots, \beta_K$ are the parameters to be estimated.

Reminder: The least squares estimates $\hat{\beta}_1, \hat{\beta}_1, \dots, \hat{\beta}_K$, are the values which minimise

$$RSS = \sum_{i=1}^n \epsilon_i^2 = \sum_{i=1}^n (y_i - \hat{f}(x_i))^2 = n \cdot MSE.$$

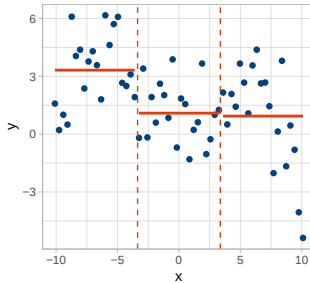
For each x_i , exactly one $C_1, \dots, C_K$ is non-zero.

The result is that $\hat{f}(x_i)$ is a piecewise constant function taking the mean values of y in each bin.

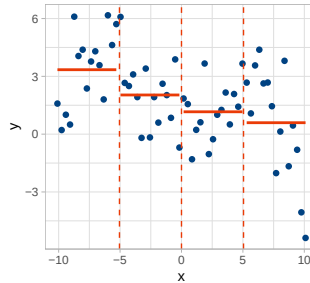


Example

Step function with 3 intervals



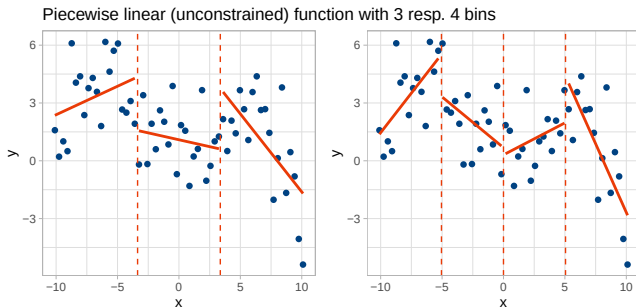
Step function with 4 intervals





Piecewise linear regression

Alternatively we could fit a piecewise linear regression in each interval:



The predictor function is now

$$f(x) = f_1(x)C_1(x) + \cdots f_K(x)C_K(x),$$

where each $f_k(x) = \beta_{0k} + \beta_{1k}x$ is a linear function.



Piecewise linear regression with constraints

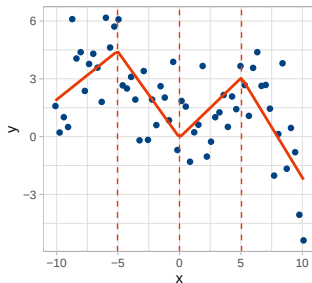
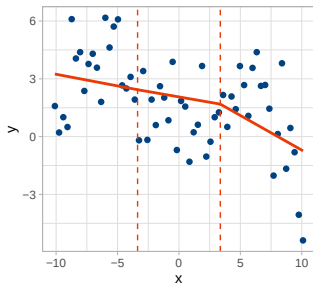
The previous diagram suffers from the problem that there is a jump (discontinuity) at the interval, which is usually inappropriate.

We can constrain the piecewise linear (pwl) function so that the function is continuous.

$$y_i = f(x_i) + \epsilon_i = f_1(x_i)C_1(x_i) + \dots + f_K(x_i)C_K(x_i) + \epsilon_i$$

where each $f_k(x_i) = \beta_{0k} + \beta_{1k}x$ **and** the boundary constraints are $f_k(c_k) = f_{k+1}(c_k)$.

Piecewise linear constrained function with 3 resp. 4 bins





Basis functions

For all of methods so far today, f can be expressed as a linear combination of basis functions.

$$y_i = f(x_i) + \epsilon_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \cdots + \beta_K b_K(x_i) + \epsilon_i,$$

where $b_k(x)$ are fixed functions called basis functions.

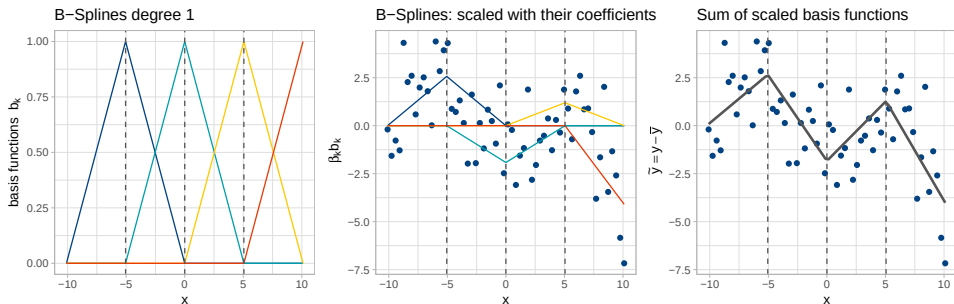
Note that there is now a constant term β_0 , which makes the fitting of the fixed basis functions easier.

- ▶ **polynomial** regression: $b_k(x) = x^k$
- ▶ **piecewise constant** functions: $b_k(x) = C_k(x_i)$



Basis functions: piecewise linear regression

In the constrained piecewise linear function we use basis functions that are in the shape of a hat **spanning two** intervals.



Remark: data is demeaned, i.e. $\tilde{y} = y - \bar{y}$ and $\hat{\beta}_0 = \bar{y}$



Piecewise linear regression (cont)

The fitted coefficients using the basis functions are

$$\beta_0 = 1.87, \beta_1 = 2.57, \beta_2 = -1.92, \beta_3 = 1.19, \beta_4 = -4.10.$$

The predictor function (right panel on previous slide) is the sum of the components:

$$\hat{f}(x) = 1.87 + 2.57 \cdot b_1(x) - 1.92 \cdot b_2(x) + 1.19 \cdot b_3(x) - 4.10 \cdot b_4(x),$$

with $b_k(x)$ being the value of the basis function at position x .

Example: prediction $\hat{f}(x_0 = 4)$

- ▶ the $b_k(4)$ with $k = 1, \dots, 4$ are

$$b_1(4) = 0.00, b_2(4) = 0.21, b_3(4) = 0.79, b_4(4) = 0.00$$

Please compare with left panel on previous slide.

- ▶ we obtain:

$$\hat{f}(4) = 1.87 + 2.57 \cdot 0.00 - 1.92 \cdot 0.21 + 1.19 \cdot 0.79 - 4.10 \cdot 0.00 = 2.41$$

- ▶ only β_0, β_2 and β_3 contribute to $\hat{f}(4)$ because $x_0 = 4$ is in bin number 3



Piecewise continuous cubic polynomials

- ▶ The constrained piecewise linear function is continuous but has sharp points at the joins, called **knots**.
- ▶ In mathematical terms the sharp points are caused because the first derivative of f is discontinuous at the knots.
- ▶ The continuity constraint was that the function value agreed on both sides of each knot¹ $f(c_k-) = f(c_k+)$.
- ▶ A smoothness constraint requires that the derivative value agrees on both sides of each knot $f'(c_k+) = f'(c_k-)$.
- ▶ This is not possible when each b_k is linear, so we specify that each piece is a polynomial function with $k > 1$.
- ▶ In practice piecewise cubic functions are fitted. These are called **cubic splines**.

¹ c_+ is $\lim x \rightarrow c_k$ from above, c_- is $\lim x \rightarrow c_k$ from below



Cubic splines

A cubic spline is a piecewise cubic function:

$$f(x) = b_k(x) \quad \text{for } c_{k-1} \leq x \leq c_k$$

with $b_k(x) = \beta_{0k} + \beta_{1k}x + \beta_{2k}x^2 + \beta_{3k}x^3$

The function is continuous, with continuous first *and* second derivatives.

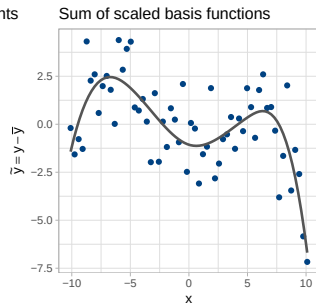
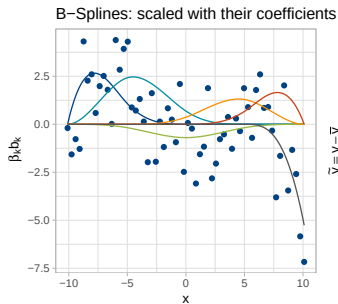
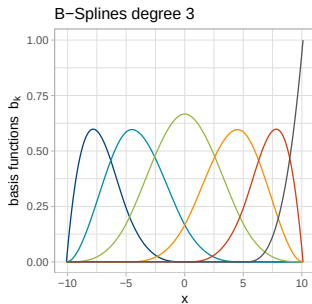
$$\begin{aligned} f(c_k-) &= f(c_k+) \\ f'(c_k-) &= f'(c_k+) \\ f''(c_k-) &= f''(c_k+) \end{aligned}$$

We only need to specify the constraints at the knots, because between the knots these conditions are a result of f being a cubic polynomial.



Cubic Splines

Remark: data is demeaned, i.e. $\tilde{y} = y - \bar{y}$ and $\hat{\beta}_0 = \bar{y}$



The cubic spline for the example data and knots at -5.05, 0.00 and 5.05.



Cubic splines

- ▶ With piecewise linear (pwl) continuous regression we expressed the function f as a linear combination of basis functions, which were “triangular hat” functions.
- ▶ With $K=3$ knots we had a constant parameter β_0 and 4 basis functions.²
- ▶ In general for polynomials of degree d and K knots, we require $K+1+d$ parameters, so for cubic splines $K+4$ parameters are needed.
- ▶ The shape of the basis functions for pwl are unique.
- ▶ For cubic splines the basis functions require another condition so that they are uniquely defined.
- ▶ The two usual methods are *B-splines* and *natural splines*.

²Note that the definition of K changes slightly here to be consistent with other texts! For spline fitting it is usual to specify K equal to the number of knots, rather than the number of bins.



B-splines

- ▶ B-splines are chosen to be efficient to compute via an iterative algorithm, details are in Hastie, Tibshirani and Friedman.
- ▶ One parameter is used for the global constant parameter so $K+3$ B-splines are required. In our example $K=3$ so we get 6 B-splines shown on slide 25 (left panel).

B-splines in R:

```
# Specifying knots
bs(x, knots = c(...), degree = 3) # degree = 3, means cubic B-splines

# Specifying degrees of freedom (df)
#
# if df is specified then the function will choose df-3 knots based on equally
# spaced quantiles of x.
bs(x, df = , degree = 3)
```



Natural splines

The other type of spline basis commonly used are the natural splines.

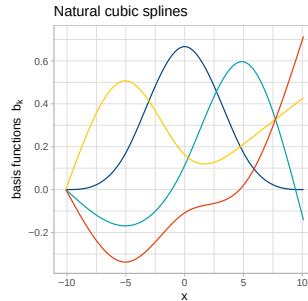
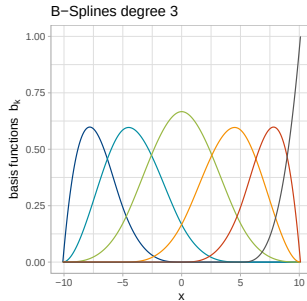
- ▶ Natural cubic splines have the *additional constraints* that the second and third derivatives are zero at the extremes of the x -data.
- ▶ If x_1 is the smallest and x_n the largest x -value then

$$f''(x_1) = f'''(x_1) = f''(x_n) = f'''(x_n) = 0.$$

- ▶ The additional constraints at the edges means that we now require only $K+1$ basis functions.



Natural splines vs B-splines

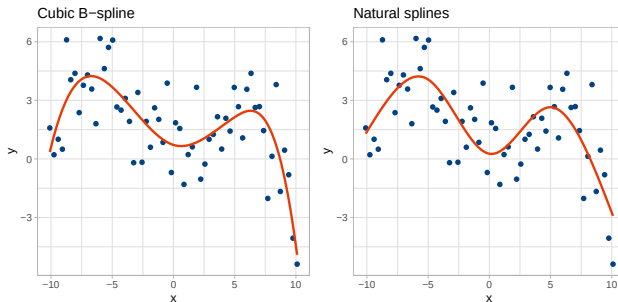


The basis functions are less visually appealing, but there is a data analytical advantage to using natural splines (next slide).



Natural splines vs B-splines

B-splines can give a predictor function with a large variance of the at the edges:



- ▶ have a look at the two diagram panels
- ▶ the problem is reduced with natural splines



Smoothing splines

We return to the penalised least squares concept you learnt in ridge regression last semester:

minimise: squared error of fitted model $+ \lambda \cdot$ penalty term

If we have a completely free choice of a non-linear function, we could choose a function f which fits the data extremely well.

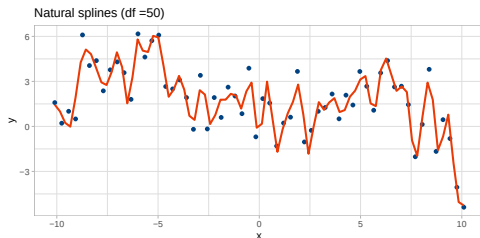
I.e. minimises squared residuals $RSS = \sum_{i=1}^n (y_i - f(x_i))^2$ almost down to 0.

A problem with this is that $f(x)$ is that this will almost certainly overfit the data. It won't *smooth* the data at all, it will interpolate the data.



Spline overfitting

Another description of the overfitting is that f is too *wiggly*:



- ▶ The wiggleness of a function is measured using the second derivative of f .
- ▶ f'' measures of how quickly the *slope* of f changes.

An overall measure of the wiggleness of our data is the integral of $f''(x)^2$ with x over the range of the predictor values: $\int_{x_{(1)}}^{x_{(n)}} f''(x)^2 dx$



Balancing over- and underfitting

To balance out the data fitting with the smoothness of the function we can use the penalised sum of squares:

$$J(f, \lambda) = \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int_{x_{(1)}}^{x_{(n)}} f''(x)^2 dx.$$

The aim is to minimise $J(f, \lambda)$ using an appropriate choice of λ .

Remarks:

- ▶ f can be any possible function as long as it is twice differentiable.
- ▶ If $\lambda = \infty$ then any wiggleness will send $J(f, \lambda)$ to infinity, so any the function that minimises J has $f''(x) = 0$.
 - ⇒ The resulting function has constant f' , so f is linear.
 - ⇒ When $\lambda = \infty$ we revert to linear regression.
- ▶ If $\lambda = 0$ then there is no penalty for wiggleness.
 - ⇒ the f that minimises $J(f, 0)$ will interpolate the data.

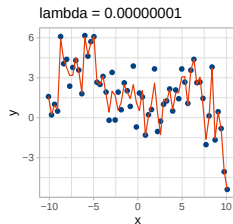
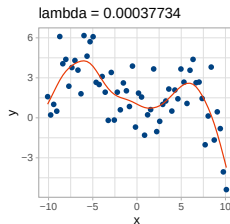
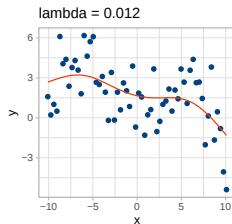
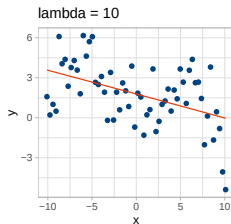


Minimizing $J(f, \lambda) \Rightarrow$ cubic spline

Theoretically one can show that for a specific λ the function $f(x)$ which minimises

$$J(f, \lambda) = \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int_{x(1)}^{x(n)} f''(x)^2 dx$$

will be a natural cubic spline!





Effective degrees of freedom

- ▶ Let N be the number of unique values of $x_1, \dots, x_n$.
- ▶ The interpolating function ($\lambda = 0$) depends on n pairs of x and y .
- ▶ A linear regression line ($\lambda = \infty$) can be defined using **two** pairs of x and y values.
- ▶ The effective degrees of freedom is a value between 2 and n which corresponds to the level of smoothing, the value of λ .



Smoothing Splines in R

In R you can specify either λ or the effective degrees of freedom. The latter is usually on a more intuitive scale.

```
smooth.spline(x = x, y = y, lambda = 0.012)
```

Call:

```
smooth.spline(x = x, y = y, lambda = 0.012)
```

Smoothing Parameter spar= NA lambda= 0.012

Equivalent Degrees of Freedom (Df): 4.008925

Penalized Criterion (RSS): 196.0342

GCV: 3.75185

```
smooth.spline(x = x, y = y, df = 4)
```

Call:

```
smooth.spline(x = x, y = y, df = 4)
```

Smoothing Parameter spar= 0.8815272 lambda= 0.01215367 (13 iterations)

Equivalent Degrees of Freedom (Df): 3.99938

Penalized Criterion (RSS): 196.3957

GCV: 3.757488



Algorithm and cross validation

- ▶ You do not need to learn the algorithm which fits the spline smoothing, but details are given in Hastie, Tibshirani & Friedman.
- ▶ The algorithm involves fixing λ and solving a set of linear equations which is an algorithm order n (i.e. fast).
- ▶ For ridge regression you learnt that cross validation is a method of choosing a good value of λ . A problem with calculating the leave one out cross validation (LOOCV) score in general is that is computationally expensive.
- ▶ With the spline smoothing algorithm, the LOOCV score can be calculated directly from the numerical solution.

Effectively we get the LOOCV score for free!



LOOCV in R

```
smooth.spline(x = x, y = y, cv = TRUE)
```

Call:

```
smooth.spline(x = x, y = y, cv = TRUE)
```

Smoothing Parameter spar= 0.6728029 lambda= 0.0003773396 (14 iterations)

Equivalent Degrees of Freedom (Df): 8.134364

Penalized Criterion (RSS): 129.1267

PRESS(1.o.o. CV): 2.889117

Compare with panels on slide 30!